

FOL: A Functional Object Lisp

Frank Adrian

Ancar Technology

Olathe, KS, USA

frank.adrian314159@gmail.com

Abstract

We present FOL (Functional Object Lisp), a Lisp dialect combining Clojure’s persistent data structures with CLOS-style class-based object orientation and a Dylan-inspired module system. We argue that persistent objects and CLOS are not merely compatible but *synergistic*: their combination yields patterns—versioned objects via the MOP, declarative event sourcing via method combinations, observable slots via predicate dispatch over change events, and lazy schema evolution that turns the traditionally difficult problem of class redefinition into a safe, implicit migration—that are more natural and concise than in either parent alone. Benchmarks show that the inherent cost of persistence is only 1–3x for sequential operations; the current 300–500x overhead is entirely attributable to interpretation. FOL is bootstrapped in Common Lisp atop the FSet and Sycamore libraries, providing Clojure-compatible sequence operations, transducers, and an adaptation of the meta-object protocol for immutable storage.

CCS Concepts

• **Software and its engineering** → **Functional languages; Object oriented languages**; *Extensible languages; Language features*.

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP, transducers

1 Introduction

Common Lisp [16] provides powerful object-oriented features through CLOS [2] and metaprogramming through the MOP [11]. Clojure [9] introduced persistent data structures emphasizing immutability. Dylan [14] explored an object-oriented Lisp with clear naming conventions. Each tradition offers distinct strengths: CLOS provides multiple dispatch, method combinations, and full metaobject protocol introspection; Clojure provides immutable data structures with $O(\log n)$ updates and structural sharing; Dylan provides clean naming and module semantics.

FOL (Functional Object Lisp) synthesizes these traditions. Our central claim is that persistent objects and CLOS are not merely compatible but *synergistic*—their combination yields patterns that are more natural and more concise than in either parent alone:

- **Versioned objects via persistence and the MOP:** Persistent snapshots with structural sharing make version chains cheap (10 lines; Section 3.5). While possible in mutable CLOS via explicit deep copies, the cost and complexity are prohibitive; Clojure has cheap snapshots but no MOP to define metaclasses that manage versioning.

- **Event sourcing via method combinations:** `:around` methods declaratively intercept all mutations to build immutable event logs, while predicate dispatch routes commands by content—15 lines in FOL versus 14 in Clojure (Section 5.2), with comparable line counts but qualitatively different guarantees: FOL’s `:around` applies automatically to all methods, while Clojure’s wrapper must be invoked explicitly.
- **Category-based predicate dispatch:** A specificity hierarchy (predicate > type > destructuring > any) that automatically orders clauses across categories while preserving definition-order within—a practical middle ground between Ernst et al.’s [6] logical implication ordering and simple first-match semantics.
- **Lazy schema evolution:** Immutability transforms the difficult problem of class redefinition into safe, implicit migration—old instances are never corrupted, and functional updates transparently produce instances of the new schema (Section 5.4).

Our benchmarks (Section 6) show that persistence adds only 1–3x overhead for sequential operations; the current 300–500x overhead reflects interpretation cost. A compiled FOL would approach native CLOS performance while retaining immutability’s correctness and concurrency benefits.

2 Language Design

2.1 Core Philosophy

FOL’s design commitments:

- (1) **Immutability by default:** All objects and collections are persistent—in the sense of Driscoll et al. [4], preserving previous versions after modification, *not* database persistence—(except streams, atoms, etc.)
- (2) **Structural sharing:** Updates create new versions sharing structure with old
- (3) **Object identity:** Distinguishes version identity (eq) from value equality (=)
- (4) **Generic functions:** Multiple dispatch over destructuring patterns
- (5) **Meta-object protocol:** Introspection and extension via adapted MOP protocols (Section 3.6)

On Identity and Equality: FOL simplifies Common Lisp’s four-level equality hierarchy (eq/eq1/equal/equalp) to two levels. eq tests *reference identity*—each functional update produces a new object, so (eq alice older-alice) returns false even when both represent “Alice.” FOL’s = tests *value equality* via generic dispatch: numbers compare with standard numeric coercion ((= 1 1.0) is

true), persistent objects compare by structural comparison of storage maps (using `fset:equal?`), and strings compare by content—subsuming the roles of `eql`, `equal`, and `equalp`. The standard implication $eq \Rightarrow =$ holds. Two independently constructed objects with identical slot values are `=`, and may be used as map keys because hashing is based on storage structure, not reference identity. For logical identity tracking across versions, applications use explicit mechanisms: identifier slots, version chains (Section 3.5), or external registries.

2.2 Syntax and Readability

FOL adopts Clojure’s reader syntax with Dylan’s `<type>` naming:

```
[1 2 3]                ; Vectors
{:name "Alice" :age 30} ; Maps
#{1 2 3 4}             ; Sets

;; Destructuring with predicate tests and type constraints
(defn factorial
  ([n (<= 1))] 1)
  ([n] (* n (factorial (dec n)))))

(defn summarize [{:keys [(name <string>) (age <number>)]}]
  (str name " is " age " years old"))
```

2.3 Type System

FOL’s type hierarchy (Figure 1) begins with `<persistent-object>`, from which most primitive, collection, and user types derive. Mutable types such as streams and atoms inherit from `standard-object` instead. Persistent primitive types wrap Common Lisp equivalents in a `val` slot, accessed through `fol-val`.

3 Architecture and Implementation

3.1 Bootstrap Implementation

The bootstrap totals approximately 18,000 lines of Common Lisp (excluding tests) with 6,333 passing tests. Table 1 shows the major components.

Table 1: Bootstrap Implementation Components

File	LOC	Purpose
eval	2,806	Evaluator and special forms
seqop	3,418	Sequence operations
standard-names	3,034	Standard environment
collection	1,404	Persistent collections
reader	1,012	FOL reader/parser
mop	1,854	Metaclass and MOP protocols
primitives	1,623	Primitive type wrappers
(others)	2,849	Strings, I/O, errors, etc.

3.2 Persistent Object Protocol

User-defined classes inherit from `<persistent-object>`:

```
(defclass <person> [<persistent-object>]
  [[name :type <string>]
   [age :type <number>]])
```

```
(def alice (make <person> :name "Alice" :age 30))
(def older-alice (assoc alice :age 31)) ; Returns new instance
(:age alice) ; => 30 (unchanged)
(:age older-alice) ; => 31
```

`defclass` supports standard CLOS options (`:type`, `:initarg`, `:initform`, `:reader`, `:writer`). Slot values are stored in a persistent hash map, enabling $O(\log n)$ updates with structural sharing.

3.3 Collection Implementation

FOL wraps `FSet` [3] and Sycamore data structures for its collections:

- **Vectors** (`[1 2 3]`): 32-way branching trees supporting $O(\log_{32} n)$ random access
- **Maps** (`{:a 1 :b 2}`): Hash array mapped tries (HAMTs) with efficient key-value operations
- **Sets** (`#{1 2 3}`): Weight-balanced binary trees for ordered iteration
- **Lists**: Persistent cons cells with standard $O(1)$ prepend and $O(n)$ access

A wrapper layer transparently converts between CL and FOL persistent objects.

3.4 Runtime Representation

Closures are persistent objects containing the function body, captured environment (an `FSet` map), and optional metadata. Immutable closed-over values enable safe sharing across threads.

Atoms are one of the few exceptions to immutability (along with streams and regular-expression scanners)—mutable references to persistent values:

```
(def counter (atom 0))
(swap! counter inc) ; Atomically updates
@counter            ; Dereferences to current value
```

Atoms use per-atom locks: `swap!` acquires the lock, applies the update function, and stores the result, ensuring serialized access from multiple threads.

Garbage collection relies on Common Lisp’s GC. Old versions are collected when unreferenced; structural sharing ensures that only unshared portions are reclaimed.

3.5 Versioned Objects via Persistence

The persistent metaclass (Section 3.6) provides the foundation for a versioned object system. Because `assoc` returns a new object sharing structure with the old, version chains are cheap to construct:

```
(defclass <versioned> [<persistent-object>]
  [[version :initform 0]
   [previous :initform nil]])

(defn with-version [obj slot-name new-value]
  (-> obj
    (assoc slot-name new-value)
    (assoc :version (inc (:version obj)))
    (assoc :previous obj)))

(defn object-history [obj]
  (take-while some? (iterate :previous obj)))
```

This pattern relies on persistence (cheap structural sharing) rather than MOP protocols directly, though the MOP’s `slot-value-using-class` redirection (Section 3.6) is what makes `assoc` on objects possible. Mutable CLOS would require explicit deep copies,

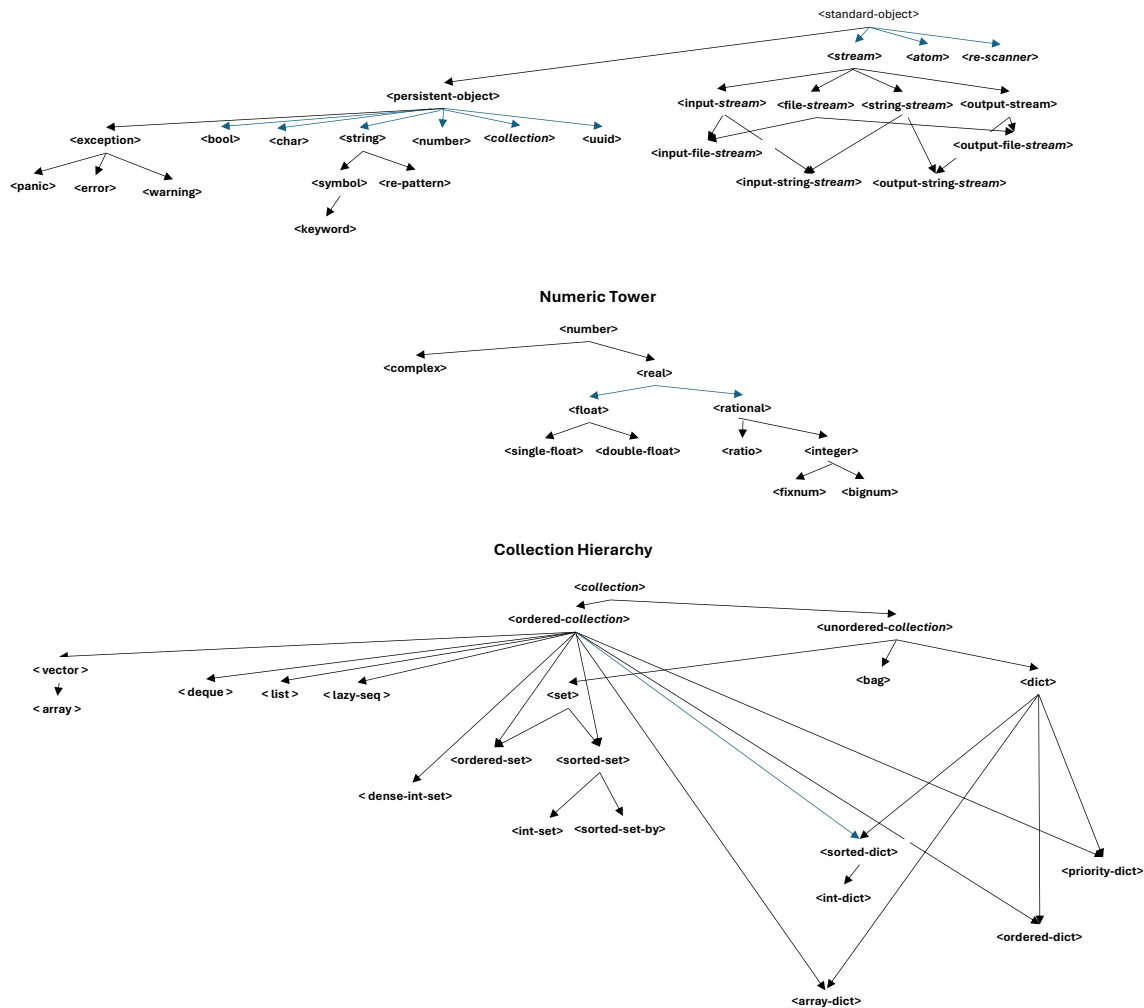


Figure 1: FOL Type Hierarchy

making version chains prohibitively expensive. Clojure maps support cheap assoc but lack metaclasses to encapsulate versioning as a reusable protocol. This enables audit trails, undo/redo, temporal queries, and debugging—all with compact storage since only changed slots require new allocations.

3.6 MOP Protocol Adaptation

FOL’s persistent metaclass adapts rather than replaces the MOP. Table 2 summarizes the protocol disposition.

Introspection protocols (class-slots, class-precedence-list, slot-definition-*) work unchanged because FOL’s class *structure* is standard CLOS—only slot *storage* is redirected. This enables tools written against the MOP to inspect FOL classes without modification.

4 Features

4.1 Clojure Features

FOL implements key Clojure abstractions:

- **Transducers** [10]: Composable transformation pipelines that work across all collection types. Transducers separate the “what” (map, filter, take) from the “how” (eager, lazy, parallel).
- **Lazy sequences**: Sequences that compute elements on demand, enabling infinite sequences and efficient streaming without mutation.
- **Thread-safe atoms**: Mutable references with per-atom locking for coordinated concurrent updates.

Table 2: MOP Protocol Adaptation

Protocol	Status	Rationale
slot-value-using-class	Redirected	Reads from FSet map
(setf slot-value-using-class)	Guarded	Blocks post-init writes
slot-boundp-using-class	Redirected	Checks FSet map membership
slot-makunbound-using-class	Blocked	Immutability invariant
validate-superclass	Extended	Cross-metaclass mixing
initialize-instance	Extended	Populates FSet map from initargs
update-instance-for-redefined-class	Omitted	Instances are immutable; lazy migration instead (Sec. 5.4)
change-class	Omitted	Violates immutability
<i>Class computation, method combination, allocation: standard CLOS behavior inherited unchanged.</i>		

- **Sequence functions:** Over 100 functions including map, filter, reduce, take, drop, partition, group-by, and frequencies.
- **Macros:** Clojure-style defmacro with syntax-quote (`'`), unquote (`~`), and splicing unquote (`~@`); macros receive unevaluated forms.
- **Error handling:** try/catch/throw forms mapped to Common Lisp’s condition system.

Metadata, refs/STM, and agents are planned.

4.2 Generic Function Integration

FOL’s generic functions dispatch on type predicates (`<vector>?`, `<dict>?`) and support multiple destructuring patterns:

```
(defgeneric process [x])
(defmethod process [(x <vector>)] (vec (map process x)))
(defmethod process [(x <dict>)] (update-vals x process))
(defmethod process [(x <number>)] (* x 2))

(process [1 {:a 2 :b 3} [[4]]]) ; => [2 {:a 4 :b 6} [[8]]]

;; Multi-clause methods with predicate dispatch
(defmethod process
  ([a] 50)
  ([a (b (< 10))] (* 2 b))
  ([a (b <number>) (c <string>)] c))

(defn is-buy-side-trade?
  ([trade (-> (trade-side) (= :buy))]) t)
  ([trade] nil))
```

Figure 2 gives the formal grammar for FOL’s multi-pattern dispatch syntax.

Predicate semantics: The form `(symbol (fn arg0 arg1 ...))` evaluates `(fn symbol arg0 arg1 ...)`—the bound value is inserted as the first argument. This works with functions, macros, or special forms.

defn vs defgeneric/defmethod: `defn` defines a standalone multi-clause function with predicate dispatch and specificity ordering; it does not create a generic function. `defgeneric` declares a generic function; `defmethod` adds methods to it. Only `defmethod` accepts `:before/`:`after/`:`around` qualifiers. Both `defn` and `defmethod` support multi-clause predicate dispatch.

4.2.1 Method Combinations. Multi-clause `defmethod` forms expand to N separate method definitions. The `:before`, `:after`, and

`:around` qualifiers apply per-clause; `call-next-method` follows CLOS semantics.

4.3 Predicate Specializers

FOL extends pattern matching with predicate specializers. The syntax `(var (fn arg0 arg1 ...))` applies the predicate to the bound value at runtime.

4.3.1 Predicate Examples. Predicates work with any function—equality, comparison, type tests, or user-defined:

```
;; Range checking with comparison predicates
(defn age-group
  ([age (< 13)]) :child)
  ([age (< 20)]) :teen)
  ([age (< 65)]) :adult)
  ([age]) :senior))

;; Type predicates and user-defined predicates
(defn process-value
  ([x (<number>?)] (* x 2))
  ([x (<string>?)] (str x x))
  ([x (<vector>?)] (vec (map process-value x)))
  ([x] x))
```

4.3.2 Predicate Specificity. FOL orders patterns by *constraint strength*—how narrowly a pattern constrains its accepted values. More specific patterns are tested first, ensuring predictable dispatch regardless of definition order:

Table 3: Predicate Specificity Levels

Level	Pattern Type	Constraint	Example
3	Predicate	Single value/narrow range	<code>(n (= 5))</code>
2	Type	All instances of a type	<code>(x <number>)</code>
1	Destructuring	Structural shape only	<code>[a b c]</code>
0	Any	Universal match	<code>x, _</code>

This ordering reflects logical subsumption: a predicate like `(= 5)` accepts strictly fewer values than `<number>`, which accepts fewer than an untyped destructuring pattern.

For sequence patterns, specificity extends to element-level comparisons: `[(x (< 0)) y]` is more specific than `[(x <number>) y]` because its first element has higher specificity.

<code>defn</code>	<code>::= (defn name (single-clause multi-clause))</code>	
<code>defgeneric</code>	<code>::= (defgeneric name (single-pattern multi-pattern))</code>	
<code>defmethod</code>	<code>::= (defmethod name qualifier[?] (single-clause multi-clause))</code>	
<code>fn</code>	<code>::= (fn (single-clause multi-clause))</code>	
<code>single-clause</code>	<code>::= pattern body⁺</code>	
<code>multi-clause</code>	<code>::= (pattern body⁺)⁺</code>	
<code>single-pattern</code>	<code>::= pattern</code>	
<code>multi-pattern</code>	<code>::= pattern⁺</code>	
<code>pattern</code>	<code>::= [param*] [param* & rest-param]</code>	
<code>param</code>	<code>::= symbol</code>	(simple binding)
	<code> (symbol type)</code>	(type specifier)
	<code> (symbol (fn-name arg*))</code>	(predicate specifier)
	<code> destructure</code>	(nested destructuring)
<code>destructure</code>	<code>::= [param*]</code>	(sequential)
	<code> { :keys [key-spec*] }</code>	(map)
<code>key-spec</code>	<code>::= symbol (symbol type) (symbol pred)</code>	
<code>type</code>	<code>::= <type-name></code>	
<code>qualifier</code>	<code>::= :before :after :around</code>	

Figure 2: FOL Multi-Pattern Dispatch Grammar (simplified)

4.3.3 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using inference rules. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p .

SPEC-LEVEL	SPEC-SEQ-HEAD
$\frac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$	$\frac{p_1 \prec q_1 \quad n \geq 1}{[p_1 \dots p_n] \prec [q_1 \dots q_n]}$
SPEC-SEQ-TAIL	
$\frac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \quad [p_2 \dots] \prec [q_2 \dots]}{[p_1 \dots p_n] \prec [q_1 \dots q_n]}$	

The operational dispatch order $\prec_{\text{disp}}$ is a total order that resolves ties (where neither $p_1 \prec p_2$ nor $p_2 \prec p_1$) using definition order. Let $\text{idx}(p)$ denote the definition index of the method clause containing p .

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge \text{idx}(p_1) < \text{idx}(p_2))$$

This lexicographic combination ensures deterministic dispatch: specificity-first, then definition-order.

4.3.4 Dispatch Semantics. Predicate specializers also work in anonymous `fn` forms but are not allowed in macro parameter lists (macros receive unevaluated forms). Internally, specializers compile to signatures of the form `(:pred fn-name (arg0 arg1 ...))`; at dispatch time, the runtime evaluates each predicate and selects the first truthy clause. Dispatch is $O(N)$ in the number of patterns at a given arity, with patterns pre-sorted by specificity. Clauses are tried in specificity order (then definition order) with short-circuit evaluation; predicate exceptions propagate immediately.

4.3.5 Type Checking. FOL's type system is entirely dynamic, with type specializers checked at runtime only. When a method specifies `(x <number>?)`, the type check occurs during dispatch via `(<number>? x)`.

```
;; No compile-time error; type mismatch detected at runtime
(defn process [(x <number>)] (* x 2))
(process "hello") ; Runtime error: does not match <number>
```

The current interpreter does not warn about unreachable clauses (e.g., a catch-all `[x]` preceding a type-specialized clause); a future compiler could detect such shadowing statically. FOL's type predicates `<number>?`, etc.) are distinct from CL's `check-type`; the `fol-val` accessor bridges these worlds.

4.3.6 Soundness Properties. FOL's dispatch maintains three invariants: (1) **exhaustive matching**—unmatched calls signal an error no-matching-clause rather than silently failing; (2) **deterministic dispatch**—the total order $\prec_{\text{disp}}$ ensures exactly one clause matches; (3) **type predicate consistency**—type specializers use the same predicates as explicit checks. FOL does *not* provide static type soundness; errors are detected at runtime only.

5 Synergy in Practice

This section demonstrates patterns that are *more natural or more concise* in FOL than in either Clojure or standard CLOS alone, because they exploit the combination of persistent data, CLOS method combinations, and predicate dispatch. Full source for all examples is available in the repository: FOL code in `fol-code/`, Common Lisp equivalents in `lisp-code/`, and Clojure equivalents in `clojure-code/`.

5.1 Code Density: Trade Compliance

We implement a trade compliance validator in FOL and plain Common Lisp (repository files `compliance.fol`, `compliance.lisp`). The FOL version uses predicate dispatch with automatic specificity ordering; the CL version uses a manual `cond` cascade with hand-ordered predicates. Table 4 compares the implementations (non-blank, non-comment lines).

The savings are modest (17%) because this example exercises predicate dispatch and module syntax but not persistence or method

Table 4: Code Comparison—Trade Compliance

Category	FOL	CL	Savings
Class definition	5	8	38%
Predicates / logic	16	13	−23%
Dispatch / validation	15	15	0%
Module boilerplate	2	10	80%
Total	38	46	17%

combinations. The predicate category is *larger* in FOL because predicate-dispatch patterns require explicit true/nil branches. The real advantage is qualitative: FOL’s specificity ordering guarantees that more-constrained rules fire first regardless of definition order, eliminating a class of maintenance bugs in the cond cascade.

5.2 Event Sourcing with Method Combinations

FOL enables event sourcing by combining `:around` methods with persistent event logs. The key mechanism is a single `:around` method that intercepts all command dispatches:

```
(defmethod apply-command :around [agg cmd]
  (bind [result (call-next-method)
        event {:command cmd :timestamp (now)}]
    (assoc result :events
            (conj (:events result) event))))
```

This `:around` method applies automatically to every `apply-command` invocation, including future methods added after deployment. The Clojure equivalent uses `defmulti/defmethod` but lacks `:around` methods, so event logging requires a manual wrapper that callers must remember to invoke. Table 5 compares the two (see repository for full source).

Table 5: Code Comparison—Event Sourcing

Category	FOL	Clj	Notes
Class / factory	2	2	Map literal
Event logging	4	4	<code>:around</code> vs wrapper
Command methods	7	5	Pred. vs key dispatch
Replay	2	2	Identical
Total	15	14	

Line counts are nearly identical, but the designs differ qualitatively. FOL’s `:around` method is *declarative*—it applies to every `apply-command` invocation automatically, including future methods added after deployment. Clojure’s wrapper requires callers to remember to use `apply-command-logged` rather than `apply-command`; calling the multimethod directly silently skips logging. This is precisely the problem that CLOS method combinations were designed to solve.

5.3 Observable Slots via MOP and Predicate Dispatch

This example combines persistent functional updates with predicate dispatch to build a change-notification system (repository

file `observable.fol`). When a slot is updated, the protocol produces both a new object (via persistent assoc) and a change event; predicate dispatch routes notifications by content—e.g., detecting reading spikes (>50 unit jump) or critical status transitions. The CL equivalent (`observable.lisp`) requires a manual copy function, a separate `defstruct` for change events, and a hand-coded cond cascade. Table 6 compares the implementations.

Table 6: Code Comparison—Observable Slots

Category	FOL	CL	Savings
Class / boilerplate	6	23	74%
Update protocol	7	9	22%
Change dispatch	18	20	10%
Total	31	52	40%

The 40% reduction comes primarily from eliminating boilerplate: FOL needs no manual copy function (persistent assoc shares structure), no separate `defstruct` for change events (persistent maps suffice), and no package ceremony. New notification rules can be added as separate `defmethod` forms without modifying the existing dispatch function, following the open/closed principle.

5.4 Lazy Schema Evolution

Standard CLOS handles class redefinition by lazily updating existing instances via `update-instance-for-redefined-class`: each instance is mutated in place the next time it is accessed. This protocol assumes mutable instances—an assumption FOL deliberately violates.

FOL’s persistent metaclass resolves this tension through what amounts to *lazy schema evolution*. Class definitions remain mutable (redefining a class updates the class object normally), but existing instances are frozen snapshots whose `%persistent-storage` FSet maps are never modified. The MOP’s `slot-value-using-class` method provides graceful degradation: when accessing a slot that was added by a redefinition but is absent from an old instance’s storage map, the method falls back to evaluating the slot’s `:initform` if one exists, or signals `slot-unbound` otherwise. Slots removed by redefinition remain in the storage map but become inaccessible (no slot definition routes to them).

Crucially, functional updates perform implicit migration of already existing objects. `set-plot-value` calls `(class-of object)` to obtain the *current* (redefined) class, allocates a fresh instance of that class, and copies the old storage map forward. The resulting object is an instance of the new schema carrying the old data—new slots resolve via `initforms`, removed slots persist harmlessly in the map.

Persistence makes this *safe*—old instances are never corrupted. In mutable CLOS, redefinition silently mutates every reachable instance; in FOL, it changes only the schema for future snapshots.

6 Evaluation

6.1 Comparison with Related Languages

FOL uniquely combines Clojure’s functional features with Common Lisp’s object system. Persistent structures incur $O(\log n)$ vs $O(1)$

Table 7: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Persistent objects	✓	✓	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Transducers	✓	✓	×
Lazy seqs	✓	✓	×
Predicate dispatch	✓	× ^a	× ^b
Macros	✓	✓	✓

^adefmulti dispatches on a single fn, not destructuring patterns.^bEQL specializers only; optima/trivia provide pattern matching as libraries.

for mutable operations, but high branching factors (32–64) keep constant factors small.

6.2 Benchmark Methodology

All measurements were taken on SBCL 2.6.0 (Windows 11, AMD Ryzen 9 5900X, 64 GB RAM). Performance benchmarks report the mean of 100 runs (coefficient of variation <2%); memory benchmarks report single allocations via `sb-ext:dynamic-space-usage`. We compare against native Common Lisp rather than Clojure: SBCL compiles to native code, providing a strict upper bound on overhead, and since FOL’s implementation is Common Lisp, this isolates abstraction cost. A Clojure comparison would conflate language design with JVM runtime differences (startup, JIT warmup, GC pauses).

6.3 Memory Benchmarks

Table 8: Vector Memory Comparison

N	CL (bytes)	FSet (bytes)	Ratio
10,000	131,008	327,472	2.50x
100,000	2,372,112	3,372,960	1.42x
1,000,000	23,982,992	33,959,040	1.42x

FSet sequences incur ~1.42x memory overhead at scale, purchasing lock-free access and cheap snapshots. Persistent objects incur ~7.3x overhead for 5-slot objects.

Table 9: Per-Slot Memory Overhead (10,000 instances)

Slots	CL bytes/inst	FSet bytes/inst	Ratio
2	216.2	236.0	1.09x
5	491.0	701.0	1.43x
10	923.0	1,707.0	1.85x
20	1,808.0	3,934.0	2.18x
50	4,455.0	13,565.0	3.04x
100	8,870.0	34,970.0	3.94x

Small objects (2–5 slots) have modest overhead (1.1–1.4x); larger objects (50–100 slots) grow to 3–4x.

Structural Sharing: Retaining 1000 versions of a 1000-element vector:

Table 10: Structural Sharing Memory (1000 versions, 1000-element vector)

Approach	Memory (bytes)	Per-version
CL (full copy each mutation)	24,000,000	24,000
FSet (structural sharing)	1,200,000	1,200
Sharing ratio	20x savings	

6.4 Performance Benchmarks

Table 11: Map Performance Comparison

N	CL (sec)	FOL (sec)	Ratio
9	0.000001	0.000263	263x
99	0.000004	0.003023	756x
999	0.000075	0.025619	342x
9,999	0.000669	0.341401	510x

The 300–500x gap reflects interpretation overhead; FSet compiled to native code achieves 2–3x of mutable. Predicate and conditional dispatch show identical performance—interpreter overhead dominates both.

6.5 Isolating Persistence from Interpretation

To separate the cost of persistence from the cost of interpretation, we benchmarked FSet operations compiled directly to native code (no FOL interpreter), averaged over 100 runs. Sequential operations (map, reduce) incur 1.1–2.7x overhead; random updates incur 21–22x ($O(\log n)$ vs $O(1)$); dictionary lookups and inserts incur 6–39x. Comparing these figures to the 300–500x overhead in the Map Performance table, we conclude that **interpretation accounts for the vast majority of overhead**, while **persistence adds only 1–3x for sequential operations**. Compilation is therefore the primary optimization target.

6.6 Threats to Validity

All measurements compare FOL’s interpreter against SBCL’s native compiler, so reported ratios reflect interpretation overhead, not inherent design costs. We do not benchmark against Clojure (see Section 6.2). Memory measurements via `sb-ext:dynamic-space-usage` may include GC artifacts. Code density comparisons use three domains; results may differ for others. FOL is a single-author implementation without independent audit.

7 Limitations and Future Work

FOL is interpreter-only; no static analysis exists—type errors and unreachable clauses are detected at runtime. Memory overhead grows with object size (1.1x for 2 slots, 4x for 100 slots). Random updates incur 21x overhead versus $O(1)$ mutation. Class redefinition does not migrate existing instances (Section 5.4); old objects retain

their original slot data, with new slots resolved lazily via `initforms`. Metadata, refs/STM, and agents are not yet implemented.

Planned extensions include **native compilation** (the primary optimization target, since interpretation dominates overhead), **abstract and sealed classes** for exhaustive pattern matching, **parallel collections** following Scala’s model, full **condition and restart support**, **STM refs** following Clojure’s `ref/dosync` model, and **channel-based concurrency** via CSP-style go blocks.

8 Related Work

Clojure [9] pioneered persistent data structures in Lisp. FOL adopts its sequence abstraction and transducers while using CLOS generic functions instead of protocols.

Common Lisp’s CLOS [2] and MOP [11] provide FOL’s object system foundation, extended with persistent slots.

Dylan [14] influenced naming conventions (`<type>` notation) and module semantics.

Persistent data structures build on Okasaki [12]; **FSet** [3] provides production-quality collections for Common Lisp. FOL’s functional update pattern is related to **lenses** in the Haskell tradition (van Laarhoven, 2009); FOL does not yet provide a lens abstraction. **Clojure’s defrecord**, Scala case classes, and Kotlin data classes all provide value-oriented objects, but none integrate with a MOP—FOL is unique in combining persistent storage with metaclass-level extensibility.

8.1 Predicate Dispatch and Pattern Matching

Ernst et al. [6] introduced predicate dispatch with logical implication for specificity. **Filtered Dispatch** [13] extended CLOS with predicate-based method selection using explicit priorities. FOL’s category-based specificity with first-match tie-breaking provides a middle ground. **Scala’s extractors** [5] and **Racket’s match expanders** provide extensible pattern matching; **OCaml’s polymorphic variants** [8] offer extensible sum types. FOL differs: predicates are arbitrary functions rather than structural extractors, and specificity is determined by category rather than type inclusion.

8.2 Multiple Dispatch and Typed Functional Lisps

Julia [1] uses multiple dispatch but is purely type-based (no predicate specializers), lacks method combinations and MOP introspection. **Coalton** [15] adds Hindley-Milner types to CL—complementary to FOL: Coalton adds static safety with mutable semantics; FOL adds persistence with dynamic typing. **Typed Racket** [19] adds gradual typing; FOL’s type specializers serve as dispatch mechanisms rather than correctness guarantees. **Carp** [17] is a statically typed Lisp with ownership semantics; it optimizes for performance where FOL optimizes for expressiveness. **Shen** [18] offers pattern matching but lacks multiple dispatch and method combinations. **Racket’s class system** [7] provides method combinations but separates classes from match; FOL unifies pattern matching and method dispatch.

9 Conclusion

FOL demonstrates that persistent objects and CLOS are synergistic rather than competing paradigms. Their combination yields versioned objects via persistent snapshots, declarative event sourcing

via method combinations over persistent logs, and observable slots via predicate dispatch over change events—patterns that are more natural and concise than in either parent language alone. Lazy schema evolution (Section 5.4) further illustrates how immutability transforms a traditionally problematic MOP protocol into a safe, implicit migration strategy.

Benchmarks confirm that interpretation, not persistence, dominates current overhead (300–500x vs 1–3x). A compiled FOL would approach native performance while retaining immutability’s correctness and concurrency benefits.

The complete implementation—385 functions, 6,333 tests, and ~18,000 lines of bootstrap code—is available at: <https://github.com/frankadrian314159/fol>

References

- [1] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.
- [2] Daniel G Bobrow, Linda G DeMichiel, Richard P Gabriel, Sonya E Keene, Gregor Kiczales, and David A Moon. 1988. Common Lisp Object System Specification. *SIGPLAN Notices* 23, SI (1988), 1–142.
- [3] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. <https://common-lisp.net/project/fset/>
- [4] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.
- [5] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007—Object-Oriented Programming*. Springer, 273–298.
- [6] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP’98—Object-Oriented Programming*. Springer, 186–211.
- [7] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.
- [8] Jacques Garrigue. 1998. Programming with Polymorphic Variants. In *ML Workshop*.
- [9] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.
- [10] Rich Hickey. 2014. Transducers. Clojure Blog. <https://clojure.org/reference/transducers>
- [11] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.
- [12] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.
- [13] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 20–26.
- [14] Andrew Shalit. 1996. *The Dylan Reference Manual*. Addison-Wesley Longman Publishing Co., Inc.
- [15] Robert Smith and Cole Bates. 2023. Coalton: Efficient, Statically Typed Functional Programming on Common Lisp. In *Proceedings of the 16th European Lisp Symposium*.
- [16] Guy L Steele Jr. 1990. *Common Lisp the Language, 2nd Edition*. Digital Press.
- [17] Erik Svedäng. 2018. Carp: A Statically Typed Lisp Without a GC. <https://github.com/carp-lang/Carp>
- [18] Mark Tarver. 2011. *The Shen Language*. Upaya Books.
- [19] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 395–406.